ring

Alice là một họa sĩ đam mê nghệ thuật trừu tượng. Hôm nay, cô ấy đang sáng tác một bức tranh khổng lồ trên một mặt phẳng giấy trắng vô hạn. Để bắt đầu, Alice dùng compa vẽ N đường tròn lên mặt giấy.

Các đường tròn này có thể nằm rời nhau, lồng vào nhau, hoặc cắt nhau chằng chịt. Mỗi khi các đường tròn giao nhau mặt phẳng giấy sẽ bị chia cắt thành nhiều mảng (miền) riêng biệt. Alice dự định sẽ tô mỗi một miền bằng một màu sắc khác nhau (bao gồm cả miền không gian rộng lớn nằm ở phía ngoài cùng của tất cả các đường tròn).

Bạn là một lập trình viên và cũng là bạn của Alice. Hãy viết một chương trình giúp Alice đếm chính xác tổng số miền trên mặt phẳng giấy để cô ấy có thể đi mua đủ số lượng màu vẽ cần thiết.

Dữ liệu vào:

- Dòng đầu tiên chứa một số nguyên dương N ($1 \leq N \leq 2000$).
- N dòng tiếp theo, mỗi dòng chứa ba số nguyên X_i, Y_i, R_i ($-10^4 \leq X_i, Y_i, R_i \leq 10^4$) tương ứng là tọa độ tâm và bán kính của đường tròn thứ i. *Không có bất kỳ hai đường tròn nào trùng nhau.*

Dữ liệu ra:

- In ra một số nguyên duy nhất: Tổng số miền (vùng) mà mặt phẳng bị chia cắt thành.

Subtask 1: $N \leq 20$;

Subtask 2: $N \leq 2000$;

Input	Output
2 0 0 1 5 0 1	3

```
const double EPS = 1e-8;

struct Point {
    double x, y;
    bool operator<(const Point& o) const {
        if (abs(x - o.x) > EPS) return x < o.x;
        return y < o.y - EPS;
    }
    bool operator==(const Point& o) const {
        return abs(x - o.x) <= EPS && abs(y - o.y) <= EPS;
    }
};

struct Circle {
    double x, y, r;
};

vector<Point> getIntersections(const Circle& c1, const Circle& c2) {
    double d = hypot(c1.x - c2.x, c1.y - c2.y);
    if (d > c1.r + c2.r + EPS || d < abs(c1.r - c2.r) - EPS || d < EPS) return {};
    double a = (c1.r * c1.r - c2.r * c2.r + d * d) / (2 * d);
    double h = sqrt(max(0.0, c1.r * c1.r - a * a));
    double cx = c1.x + a * (c2.x - c1.x) / d;
    double cy = c1.y + a * (c2.y - c1.y) / d;
    Point p1 = {cx + h * (c1.y - c2.y) / d, cy + h * (c2.x - c1.x) / d};
    Point p2 = {cx - h * (c1.y - c2.y) / d, cy - h * (c2.x - c1.x) / d};
    if (abs(h) < EPS) return {p1};
    return {p1, p2};
}
```